# Programming 1: Lab 6 : Loops

**Write the python code for the following questions. Handle all the valid and invalid test cases. Write down relevant comments in your code:**

1. Display the following addition table:

```
              Addition Table
         1   2   3   4   5   6   7   8   9
    ----------------------------------------
  1 |   2   3   4   5   6   7   8   9  10
  2 |   3   4   5   6   7   8   9  10  11
  3 |   4   5   6   7   8   9  10  11  12
  4 |   5   6   7   8   9  10  11  12  13
  5 |   6   7   8   9  10  11  12  13  14
  6 |   7   8   9  10  11  12  13  14  15
  7 |   8   9  10  11  12  13  14  15  16
  8 |   9  10  11  12  13  14  15  16  17
  9 |  10  11  12  13  14  15  16  17  18
```

2. **Parking Slot Allotment.** A parking area has slots numbered from 1 to 1000. Every slot whose number is divisible by N is reserved. However, a reserved slot whose number is also divisible by 4 is kept for electric vehicle charging and is not given to staff. Display the slot numbers given to the staff. Use the continue statement with a while loop.

   **Input:** A single positive integer N.
   **Output:** The slot numbers given to the staff, separated by a single space.
   **Constraints:** 1 ≤ N ≤ 1000.

| Sample Input | Sample Output |
|---|---|
| 250 | 250 750 |
| 333 | 333 666 999 |

   *For N = 250 the reserved slots are 250, 500, 750 and 1000, out of which 500 and 1000 are kept for charging.*

3. Display the following patterns for N lines (value of N taken from the user) using while loop:

```
a)   1
     1 2
     1 2 3
     1 2 3 4

     (till N lines)
```

```
b)   * * * *
     * * *
     * *
     *

     (till N lines)
```

```
c)         *
         * *
       * * *
     * * * *

     (till N lines)
```

4. Take a positive integer as input from the user and display its octal equivalent via the repeated division method. (without using any inbuilt function or format specifier). The final result should be displayed as a string. Check if the result matches the output of inbuilt oct() function.

5. **Sports Day Javelin Throw.** N students take part in the javelin throw event and the distance thrown by each of them is recorded. Find the winning distance and the runner-up distance, where the runner-up distance is the largest distance that is strictly smaller than the winning one. Do not use any inbuilt function.

   **Input:** The first line contains the integer N. The next N lines contain one distance each (a positive integer, in metres).
   **Output:** Print the winning distance in the first line and the runner-up distance in the second line.
   **Constraints:** 2 ≤ N ≤ 100. At least two different distances are recorded.

| Sample Input | Sample Output |
|---|---|
| 5<br>42<br>55<br>55<br>38<br>47 | 55<br>47 |

*55 is thrown twice, so the runner-up distance is 47 and not 55.*

6. Display all the Armstrong numbers between 1 and N, where the value of N is taken as input from the user.

7. Print the following table to display the **square**, **cube** and **square root** of the numbers from 1 to 15. Round the value to keep four digits after the decimal point.

```
N      N^2      N^3      Square Root
1        1        1        1.0000
2        4        8        1.4142
3        9       27        1.7321
...
14     196     2744        3.7417
15     225     3375        3.8730
```

8. Display the following pattern using nested loops:

```
   1
  2 2
 3 3 3
4 4 4 4
(till N lines)
```

9. Display the following triangle of consecutive numbers using nested loops:

```
1
2 3
4 5 6
7 8 9 10
(till N lines)
```

10. Take an odd positive integer N as input and display the following diamond pattern of N lines using nested loops:

```
  *
 ***
*****
 ***
  *
```
(shown for N = 5)